

Programação Orientada a Objetos

Relacionamentos entre Classes no GestorOS

[Baixe aqui os exemplos da aula](#)



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Associação: Referência entre Objetos

Uma associação existe quando um objeto guarda, entre seus atributos, uma referência a outro objeto, e não apenas um dado que o descreve de longe. Equipamento passa a declarar `private Cliente cliente` em vez de um número solto: o compilador garante que ali exista, de fato, um objeto `Cliente` construído e válido, não um inteiro cujo significado depende de uma tabela externa.



Cliente Sobrevive ao Equipamento

A associação liga dois objetos com ciclos de vida independentes: o cliente já existia antes de o equipamento ser cadastrado e continuará existindo se o equipamento for descartado, devolvido ao dono ou simplesmente esquecido no sistema. Nada nessa relação obriga um dos dois a desaparecer junto do outro. Equipamento apenas referencia Cliente; não o possui, não o cria e não o destrói.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Equipamento Referencia Cliente



A seta liga sem possuir: removido o Equipamento, o **Cliente continua existindo**.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Referência de Objeto no Lugar do Id Solto

Guardar `idCliente` como inteiro não impede um equipamento de apontar para um cliente inexistente: o número 47 compila e roda mesmo que nenhum cliente com esse identificador tenha sido criado. Guardar `Cliente cliente` elimina essa classe inteira de erro, porque o próprio compilador exige um objeto `Cliente` já construído antes de aceitar a atribuição — a integridade deixa de depender de disciplina externa.



Composição: o Todo Cria as Partes

Composição é o relacionamento mais forte entre objetos: a parte não tem existência própria fora do todo que a contém. ItemServico nasce dentro de uma OrdemServico, por ela é criado e adicionado, e não carrega sentido de domínio isolado — não existe tela nem consulta que liste itens de serviço soltos, sem uma ordem que os agrupe e justifique sua existência.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

OrdemServico Compõe os Itens



Os itens vivem **dentro da moldura** da ordem: sem OrdemServico, nenhum ItemServico existe.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Associação e Composição: Ciclos Opostos

A diferença entre os dois relacionamentos está inteira no ciclo de vida. Cliente sobrevive ao Equipamento porque a associação apenas liga; ItemServico não sobrevive à OrdemServico porque a composição também possui. A mesma pergunta separa os dois casos: destruído o objeto que guarda a referência, o objeto referenciado ainda faz sentido sozinho? Para Cliente, sim; para ItemServico, não.



Lista Encapsulada: Cópia, Não a Original

Expor a lista itens diretamente permitiria que qualquer código externo inserisse ou removesse elementos sem passar por adicionarItem, quebrando a composição na prática mesmo que ela exista na intenção do projetista. getItens() devolve List.copyOf(itens), uma cópia imutável: quem consulta os itens os enxerga inteiros, mas não pode alterá-los por fora da classe que os compõe.



Um Objeto, Dois Relacionamentos

OrdemServico participa dos dois relacionamentos ao mesmo tempo, cada um com seu próprio significado. Ela associa-se a Cliente e a Equipamento, objetos que preexistem e sobrevivem independentemente da ordem; e compõe uma lista de ItemServico, objetos que ela mesma cria e que desaparecem se ela desaparecer. Reconhecer qual relacionamento está em jogo evita tratar posse e vínculo como sinônimos.



calcularTotal: a Primeira Regra de Negócio

Até aqui o modelo apenas guardava e validava dados; calcularTotal é a primeira regra de negócio que nasce diretamente de um relacionamento. O método percorre a lista de ItemServico e soma cada valor com BigDecimal.add, nunca com o operador +, que double aceitaria sem avisar do erro de arredondamento. A composição deixa de ser só estrutura e passa a sustentar cálculo.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Equipamento: Construtor Exige um Cliente

```
private Cliente cliente;

public Equipamento(Cliente cliente, String tipo, String marca,
                    String defeito) {
    if (cliente == null) {
        throw new IllegalArgumentException("Equipamento precisa de um cliente.");
    }
    this.cliente = cliente;
    setTipo(tipo);
    this.marca = marca;
    setDefeito(defeito);
}
```



OrdemServico: Composição e Soma dos Itens

```
private final List<ItemServico> itens = new ArrayList<>();

public void adicionarItem(ItemServico item) {
    // O item nasce aqui: nao ha construtor de ItemServico fora da OS.
    if (item == null) throw new IllegalArgumentException("Item nulo.");
    itens.add(item);
}

public BigDecimal calcularTotal() {
    BigDecimal total = BigDecimal.ZERO;
    for (ItemServico item : itens) {
        total = total.add(item.getValor());
    }
    return total;
}
```



GestorOS: Associação e Composição em Ação

```
Cliente cliente = new Cliente("Marina Alvarenga", "(32) 98811-4402");

// Associacao: o equipamento guarda uma referencia direta ao cliente.
Equipamento equip = new Equipamento(cliente, "Notebook", "Acer",
                                         "Nao liga apos queda de energia");

// Composicao: os itens abaixo so existem porque esta OS existe.
OrdemServico os = new OrdemServico(cliente, equip);
os.adicionarItem(new ItemServico("Fonte 65W", new BigDecimal("149.90")));
os.adicionarItem(new ItemServico("Mao de obra", new BigDecimal("80.00")));

System.out.println("Total: R$ " + os.calcularTotal());
```



Atividades

Exercício 1

Por que Equipamento agora exige um Cliente não nulo no construtor, em vez de aceitar qualquer inteiro maior que zero como fazia até a Aula 02?

Exercício 2

Explique, com as próprias palavras, por que Cliente sobrevive ao Equipamento mas ItemServico não sobrevive à OrdemServico.

Exercício 3

Por que getItens() devolve List.copyOf(itens) em vez da lista itens diretamente? O que mudaria se devolvesse a lista original?



Desafio extra

Opcional

Quem quiser ir além do combinado nesta semana:

1. Adicionar a Peca como um segundo tipo de item possível dentro de OrdemServico, reaproveitando a composição já existente.
2. Criar um método cancelar() em OrdemServico que muda o status para CANCELADA e impede novos adicionarItem.

Nada disso é cobrado na entrega. O projeto segue completo sem essas adições.

